Lab 6 Git and GitHub for Software Development Management

Five short exercises | About 30 minutes

Use Git to manage a small SaaS project's scope and release planning documents. Create two versions, synchronize a team update, merge a release checklist and undo one temporary mistake.

Submission

Submit your public GitHub repository URL and one PDF report with exactly five short answers and five screenshots. Use the supplied report template.

Before the timed lab

Have Git installed and a GitHub account ready. Use Git Bash on Windows so the commands in this handout match your terminal. Complete any first-time GitHub sign-in prompt before starting. Installation and account setup are outside the 30-minute activity.

Create an empty PUBLIC GitHub repository named SEG4105_Lab6 in your own account: click the + menu, New repository, enter the name, choose Public and click Create repository. Leave README, .gitignore and license initialization unchecked.

Create an empty folder named SEG4105_Lab6 on your computer. Open that folder in File Explorer, right-click its blank area and choose Open Git Bash here. Open the same folder in your text editor. All local commands and edits below stay in this one folder.

When creating README.md or release-checklist.md, use those exact names. In Windows Notepad, choose Save as type All files so the file does not become README.md.txt. Save file edits before running Git commands.

How to follow the handout

Text labelled Editor is typed into the named file, not into the terminal. Text labelled Terminal is run in Git Bash. Replace YOUR-USERNAME, YOUR-NAME and YOUR-GIT-EMAIL with your own values. The commands do not include a prompt symbol to copy.

Suggested pace: Exercises 1 to 5 take about 6, 5, 6, 5 and 3 minutes, followed by 5 minutes to finish the short report.

Public instructor example: https://github.com/Zwang-23/SEG4105_Lab6. The example is already completed; create your own empty repository for the exercises.

Exercise 1 Set up a shared project repository

Repository Setup Pipeline | About 6 minutes

Management purpose: establish a shared starting baseline for the project.

1 Editor create README md

In your local SEG4105_Lab6 folder, create README.md. Type these three lines exactly and save the file.

```
# Tenant Portal
Version: v1
Scope: sign-in and dashboard.
```

2 Terminal initialize and commit

```
git init -b main
git config user.name "YOUR-NAME"
git config user.email "YOUR-GIT-EMAIL"
git add README.md
git commit -m "BASELINE: record project scope v1"
```

3 Terminal connect and upload

Replace YOUR-USERNAME in this URL with your GitHub username. Use the empty repository you created before the lab.

```
git remote add origin https://github.com/YOUR-USERNAME/SEG4105_Lab6.git
git push -u origin main
```

4 Browser view the public baseline

Open your repository page and refresh it. It should display README.md and the three lines you saved. Capture S1 now.

Check the result

The first commit is on GitHub and the repository is public. Your local folder remains the working folder for Exercise 2.

Answer in one or two sentences

Why is a shared versioned repository useful to a software development team?

Take one screenshot

S1: your GitHub repository page after the first push. Show the public repository name and README content in the same screenshot.

Exercise 2 Record a scope change

Local Staging Pipeline | About 5 minutes

Management purpose: record change request CR01, adding CSV export to the project scope.

1 Editor update README md

Open the existing local README.md. Replace its entire contents with these four lines and save. Do not edit the GitHub webpage in this exercise.

```
# Tenant Portal
Version: v2
Scope: sign-in, dashboard, and CSV export.
Change CR01: CSV export requested by the product owner.
```

2 Terminal inspect then record the change

```
git status
git diff -- README.md
git add README.md
git commit -m "CR01: add CSV export to project scope"
git push
```

3 Terminal compare the two versions

```
git log --oneline -2
git diff HEAD~1 HEAD -- README.md
```

HEAD is the latest commit; HEAD~1 is its previous commit here. The comparison should show Version changing from v1 to v2 and CSV export being added. Capture S2 after these two commands.

Check the result

The history contains the baseline and the CR01 commit. Both versions remain retrievable even though README.md currently displays v2.

Answer in one or two sentences

How does a descriptive commit message help a manager track a scope change?

Take one screenshot

S2: one terminal screenshot showing git log --oneline -2 and the v1 to v2 diff.

Exercise 3 Pull and push a team update

Collaboration Pipeline | About 6 minutes

Management purpose: exchange an update about who will test the new feature. You will simulate a teammate by editing on GitHub.

1 Browser add the team update

On YOUR repository page, click README.md, then the pencil button Edit this file. Add this exact new line at the end of the file:

```
Team update: QA will test CSV export.
```

Click Commit changes. Enter the message "TEAM: assign CSV export testing to QA", select Commit directly to the main branch and confirm Commit changes. This changes GitHub; your local file is still the earlier copy.

2 Terminal fetch and pull

```
git fetch origin
git pull --ff-only
```

Reopen or reload the local README.md in your editor. The Team update line should now be present. Fetch downloads remote information; pull updates your current branch and file. No local edit should be made before this pull.

3 Editor add the manager update

At the end of the local README.md, add this exact line and save:

```
Manager update: CSV export testing assigned to QA.
```

4 Terminal commit and push

```
git add README.md
git commit -m "PLAN: confirm CSV export testing responsibility"
git push
```

Capture S3 with the pull and push success messages visible in one terminal screenshot. Refresh GitHub and check that both update lines are present.

Check the result

The local file received the team update from GitHub, and GitHub received the manager update from your local commit.

Answer in one or two sentences

Why should a team member pull the shared project plan before continuing work?

Take one screenshot

S3: one terminal screenshot showing the successful pull and push output. Scroll or resize the terminal so both are readable.

Exercise 4 Merge a release checklist

Feature Lifecycle Pipeline | About 5 minutes

Management purpose: prepare release checks on a branch and integrate them into the shared project.

1 Terminal create the feature branch

```
git switch -c feature/release-checklist
```

2 Editor create release checklist md

In the same local folder, create a NEW file named release-checklist.md. Type these four lines and save:

```
# Release checklist
- Test sign-in.
- Test CSV export.
- Release manager checks test results.
```

3 Terminal commit and merge

```
git add release-checklist.md
git commit -m "FEATURE: add release checklist"
git switch main
git merge --no-ff --no-edit feature/release-checklist
git push
```

The merge options create a visible merge commit and use its default message without opening a text editor. No conflict is expected because only the feature branch added this file.

4 Terminal show the integration

```
git log --graph --oneline --decorate -5
cat release-checklist.md
```

Capture S4. cat displays the file contents in Git Bash. Refresh GitHub to see release-checklist.md on main.

Check the result

The branch graph records the integration, and main contains the checklist. The checklist states planned checks; this lab does not execute application tests.

Answer in one or two sentences

Why use a separate branch when preparing a release checklist?

Take one screenshot

S4: one terminal screenshot showing the branch graph and release-checklist.md content.

Exercise 5 Undo an uncommitted edit

Correction Pipeline | About 3 minutes

Management purpose: inspect and undo a mistaken document edit before it enters the shared baseline.

1 Editor add one temporary line

Open the LOCAL README.md in the same folder. Go to the end of the file and add exactly this new line, then save:

```
TEMP: accidental edit for Lab 6.
```

For this exercise, do not run git add, git commit or git push after adding the temporary line.

2 Terminal inspect the edit

```
git diff -- README.md
```

You should see a line beginning with +TEMP: accidental edit for Lab 6. The plus sign is produced by Git to mark added text; it is not part of the line you typed.

3 Terminal discard just that file edit

```
git restore README.md
cat README.md
git status --short
```

Reopen README.md if needed. The TEMP line should be gone, while the earlier scope and team updates remain. git status --short should print nothing because the working folder is clean. Capture S5 with the earlier diff and this result visible in one terminal view.

Check the result

The temporary edit was discarded. The earlier committed project records and their history remain intact.

Answer in one or two sentences

Why should a team inspect a document change before discarding it?

Take one screenshot

S5: one terminal screenshot showing the temporary line in git diff, followed by the restored README and clean status.